a. The two Generative AIs I used for this assignment are Gemini and ChatGPT.
b. I accessed both AIs from their respective sites: gemini.google.com and chatgpt.com
c. The prompt I used for both generated code is as following:

Generate C++ code using objects, not functions, to implement
the following requirements:
• Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
o Each digit from 1 to 9 must appear exactly once in each row.
o Each digit from 1 to 9 must appear exactly once in each column.
o Each digit from 1 to 9 must appear exactly once in each block.
• Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
• The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
• That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And

by “slower to run,” we mean it'll finish in a few tenths of a second rather than a few milliseconds.)

- Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
- This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
- You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
- Write a recursive object-oriented program to find the solution to the five puzzle files.
- The output for each solution, should include:
  - o The puzzle file name (e.g., puzzle1.txt)
  - o The puzzle from the file printed out.
  - o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
- The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
- To help you debug your program, solution1.txt is a solution to puzzle1.txt.

---

d. Below are the two generated codes:

ChatGPT's code :

```
#include <array>
#include <cstdint>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

class SudokuPuzzle {
public:
    static constexpr int Size = 9;
    using Grid = std::array<std::array<int, Size>, Size>;

    explicit SudokuPuzzle(std::string fileName)
        : fileName_(std::move(fileName)) {
        load();
    }

    void printPuzzle(std::ostream& output) const {
        output << "Puzzle file: " <<
std::filesystem::path(fileName_).filename().string()
               << "\nPuzzle:\n";
        printGrid(original_, output, true);
    }

    void solveAndPrintAll(std::ostream& output) {
        solutionCount_ = 0;
        output_ = &output;
        if (valid_) {
            solveRecursively();
        }

        if (solutionCount_ == 0) {
            output << "No solution found\n";
```

```
        } else {
            output << "Solutions found: " << solutionCount_ << "\n";
        }
    }

private:
    static constexpr std::uint16_t AllDigitsMask = 0x3FE; // Bits 1 through 9.

    std::string fileName_;
    Grid original_{};
    Grid grid_{};
    std::array<std::uint16_t, Size> rowMasks_{};
    std::array<std::uint16_t, Size> columnMasks_{};
    std::array<std::uint16_t, Size> blockMasks_{};
    std::size_t solutionCount_ = 0;
    std::ostream* output_ = nullptr;
    bool valid_ = true;

    static int blockIndex(int row, int column) {
        return (row / 3) * 3 + (column / 3);
    }

    void load() {
        std::ifstream input(fileName_);
        if (!input) {
            throw std::runtime_error("Could not open " + fileName_);
        }

        std::vector<char> cells;
        char symbol = '\0';
        while (input.get(symbol)) {
            if (symbol == '_' || (symbol >= '1' && symbol <= '9')) {
                cells.push_back(symbol);
            } else if (symbol != ' ' && symbol != '\t' &&
                       symbol != '\r' && symbol != '\n') {
                throw std::runtime_error(
                    "Invalid character in " + fileName_ + ": " + symbol);
            }
        }

        if (cells.size() != Size * Size) {
            throw std::runtime_error(
                fileName_ + " must contain exactly 81 cells");
```

```
        }

        for (int row = 0; row < Size; ++row) {
            for (int column = 0; column < Size; ++column) {
                const char cell = cells[row * Size + column];
                const int value = (cell == '_') ? 0 : cell - '0';
                original_[row][column] = value;
                grid_[row][column] = value;

                if (value != 0) {
                    addGiven(row, column, value);
                }
            }
        }
    }

    void addGiven(int row, int column, int value) {
        const std::uint16_t bit = static_cast<std::uint16_t>(1u << value);
        const int block = blockIndex(row, column);

        if ((rowMasks_[row] & bit) || (columnMasks_[column] & bit) ||
            (blockMasks_[block] & bit)) {
            valid_ = false;
        }

        rowMasks_[row] |= bit;
        columnMasks_[column] |= bit;
        blockMasks_[block] |= bit;
    }

    std::uint16_t candidatesFor(int row, int column) const {
        const std::uint16_t used = static_cast<std::uint16_t>(
            rowMasks_[row] | columnMasks_[column] |
            blockMasks_[blockIndex(row, column)]);
        return static_cast<std::uint16_t>(AllDigitsMask & ~used);
    }

    static int candidateCount(std::uint16_t candidates) {
        int count = 0;
        for (int value = 1; value <= Size; ++value) {
            if (candidates & static_cast<std::uint16_t>(1u << value)) {
                ++count;
            }
```

```
        }
        return count;
    }

    bool chooseEmptyCell(int& chosenRow, int& chosenColumn,
                         std::uint16_t& candidates) const {
        int fewestCandidates = Size + 1;
        bool foundEmptyCell = false;

        for (int row = 0; row < Size; ++row) {
            for (int column = 0; column < Size; ++column) {
                if (grid_[row][column] != 0) {
                    continue;
                }

                foundEmptyCell = true;
                const std::uint16_t current = candidatesFor(row, column);
                const int count = candidateCount(current);

                if (count < fewestCandidates) {
                    fewestCandidates = count;
                    chosenRow = row;
                    chosenColumn = column;
                    candidates = current;

                    if (count <= 1) {
                        return true;
                    }
                }
            }
        }

        return foundEmptyCell;
    }

    void place(int row, int column, int value) {
        const std::uint16_t bit = static_cast<std::uint16_t>(1u << value);
        grid_[row][column] = value;
        rowMasks_[row] |= bit;
        columnMasks_[column] |= bit;
        blockMasks_[blockIndex(row, column)] |= bit;
    }
```

```
    void remove(int row, int column, int value) {
        const std::uint16_t bit = static_cast<std::uint16_t>(1u << value);
        grid_[row][column] = 0;
        rowMasks_[row] &= static_cast<std::uint16_t>(~bit);
        columnMasks_[column] &= static_cast<std::uint16_t>(~bit);
        blockMasks_[blockIndex(row, column)] &= static_cast<std::uint16_t>(~bit);
    }

    void solveRecursively() {
        int row = -1;
        int column = -1;
        std::uint16_t candidates = 0;

        if (!chooseEmptyCell(row, column, candidates)) {
            ++solutionCount_;
            *output_ << "Solution " << solutionCount_ << ":\n";
            printGrid(grid_, *output_, false);
            return;
        }

        for (int value = 1; value <= Size; ++value) {
            const std::uint16_t bit = static_cast<std::uint16_t>(1u << value);
            if ((candidates & bit) == 0) {
                continue;
            }

            place(row, column, value);
            solveRecursively();
            remove(row, column, value);
        }
    }

    static void printGrid(const Grid& grid, std::ostream& output,
                          bool showBlanks) {
        for (const auto& row : grid) {
            for (int column = 0; column < Size; ++column) {
                if (column != 0) {
                    output << ' ';
                }
                if (row[column] == 0 && showBlanks) {
                    output << '_';
                } else {
                    output << row[column];
```

```
                }
            }
            output << '\n';
        }
        output << '\n';
    }
};

class SudokuApplication {
public:
    int run(int argumentCount, char* arguments[]) const {
        std::vector<std::string> fileNames;
        if (argumentCount > 1) {
            for (int index = 1; index < argumentCount; ++index) {
                fileNames.emplace_back(arguments[index]);
            }
        } else {
            fileNames = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt",
                         "puzzle4.txt", "puzzle5.txt"};
        }

        bool hadError = false;
        for (const std::string& fileName : fileNames) {
            try {
                SudokuPuzzle puzzle(fileName);
                puzzle.printPuzzle(std::cout);
                puzzle.solveAndPrintAll(std::cout);
                std::cout << '\n';
            } catch (const std::exception& error) {
                hadError = true;
                std::cerr << "Error: " << error.what() << "\n\n";
            }
        }

        return hadError ? 1 : 0;
    }
};

int main(int argc, char* argv[]) {
    const SudokuApplication application;
    return application.run(argc, argv);
}
```

Gemini's Code:

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <fstream>

class SudokuBoard {
private:
    int grid[9][9];

    // Check if placing num at (row, col) is valid
    bool isValid(int row, int col, int num) const {
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i] == num) return false;
            if (grid[i][col] == num) return false;
        }

        int startRow = row - row % 3;
        int startCol = col - col % 3;
        for (int r = 0; r < 3; ++r) {
            for (int c = 0; c < 3; ++c) {
                if (grid[startRow + r][startCol + c] == num) return false;
            }
        }
        return true;
    }

    // Recursive backtracking solver that finds all solutions
    bool solveRecursive(int row, int col, int& solutionCount) {
        if (row == 9) {
            solutionCount++;
            std::cout << "Solution #" << solutionCount << ":\n";
            printGrid();
            return true; // Continue searching for other potential solutions
        }

        if (col == 9) {
            return solveRecursive(row + 1, 0, solutionCount);
        }
```

```
        if (grid[row][col] != 0) {
            return solveRecursive(row, col + 1, solutionCount);
        }

        bool found = false;
        for (int num = 1; num <= 9; ++num) {
            if (isValid(row, col, num)) {
                grid[row][col] = num;
                if (solveRecursive(row, col + 1, solutionCount)) {
                    found = true;
                }
                grid[row][col] = 0; // Backtrack
            }
        }
        return found;
    }

    void printGrid() const {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                if (grid[i][j] == 0) std::cout << "_ ";
                else std::cout << grid[i][j] << " ";
            }
            std::cout << "\n";
        }
        std::cout << "\n";
    }

public:
    SudokuBoard() {
        clear();
    }

    void clear() {
        for (int i = 0; i < 9; ++i)
            for (int j = 0; j < 9; ++j)
                grid[i][j] = 0;
    }

    // Load puzzle from file (expects digits 1-9 and '_' or '0' for blanks)
    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
```

```
        if (!file.is_open()) return false;

        char ch;
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                if (!(file >> ch)) return false;
                if (ch == '_' || ch == '0') {
                    grid[i][j] = 0;
                } else if (ch >= '1' && ch <= '9') {
                    grid[i][j] = ch - '0';
                } else {
                    --j; // Retry if whitespace/formatting artifact
                }
            }
        }
        return true;
    }

    // Print original puzzle and execute solver
    void solve(const std::string& filename) {
        std::cout << "========================================\n";
        std::cout << "Puzzle File: " << filename << "\n";
        std::cout << "========================================\n";

        if (!loadFromFile(filename)) {
            std::cout << "Error: Could not open file or invalid format.\n\n";
            return;
        }

        std::cout << "Original Puzzle:\n";
        printGrid();

        int solutionCount = 0;
        std::cout << "Solving...\n\n";
        solveRecursive(0, 0, solutionCount);

        if (solutionCount == 0) {
            std::cout << "No solution found\n";
        } else {
            std::cout << "Total solutions found: " << solutionCount << "\n";
        }
        std::cout << "\n";
    }
```

```
};

int main() {
    SudokuBoard solver;

    // Process puzzle1.txt through puzzle5.txt
    for (int i = 1; i <= 5; ++i) {
        std::string filename = "puzzle" + std::to_string(i) + ".txt";
        solver.solve(filename);
    }

    return 0;
}
```

e. In terms of correctness, both programs follow the instructions. They are object-oriented, both solve the sudoku puzzles through backtracking, can read the files, and print out the puzzles and solutions. For input validation, ChatGPT scans for 81 cells, duplicates, and responds to file errors with a printed message. Gemini fails to handle inputs like invalid puzzles with duplicates or more than 81 cells. One edge case Gemini is able to handle is 0s while ChatGPT only handles expected characters like numbers and underscores.

When evaluating execution time, both approach a worst-case time complexity of O(9^n) because they try every number for each cell. Because they have different approaches, with Gemini checking the row, column, and box for every number, and with ChatGPT tracking used numbers and choosing a cell with least possible candidates, ChatGPT will on average reach a solution faster.

Analyzing space complexity reveals that both programs use O(n) space in the stack for their recursive calls. Gemini uses less memory during its solving time because it only uses one sudoku grid, while ChatGPT stores an original grid and a solving grid. ChatGPT also tracks numbers using masks.

Gemini's code is evidently better in maintainability. It is much shorter and its syntax is easier to understand; however, it does include an unused vector library and does not print specific error messages like ChatGPT does. ChatGPT's program is more modularized and easier to debug with its error messages. Gemini has few comments while ChatGPT's code contains none.

f. The code I chose for the basis of this assignment is ChatGPT. Although its code is more complex, it is able to handle inputs better and solve puzzles quicker through its tracking. Improving ChatGPT's code will not require significant changes to its solving algorithm to be faster.

g. I improved correctness by printing an 'invalid puzzle' message when the input contains duplicate numbers in the puzzle. This allows the user to evaluate the output easier.
   Evaluation time was improved by returning the recursive call when a solution is no longer possible for an empty cell. This cuts the recursion shorter, possible saving time.
   Since ChatGPT's code contains both an original grid and the solution grid it prints, it wastes space. Removing the original grid and just printing the

grid before it begins solving will save space.

Finally, maintainability is improved by adding inline comments to all lines of the code. Additionally, initializing a constant for box sizes instead of using 3s will help readability in loops.